

Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch

```
PS C:\Users\admin\Documents\FlutterProjects\Student-List> git checkout -b feature/student-list
Switched to a new branch 'feature/student-list'
PS C:\Users\admin\Documents\FlutterProjects\Student-List> git branch
PS C:\Users\admin\Documents\FlutterProjects\Student-List>
```

🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart `List`.

Research what a Dart `List` is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

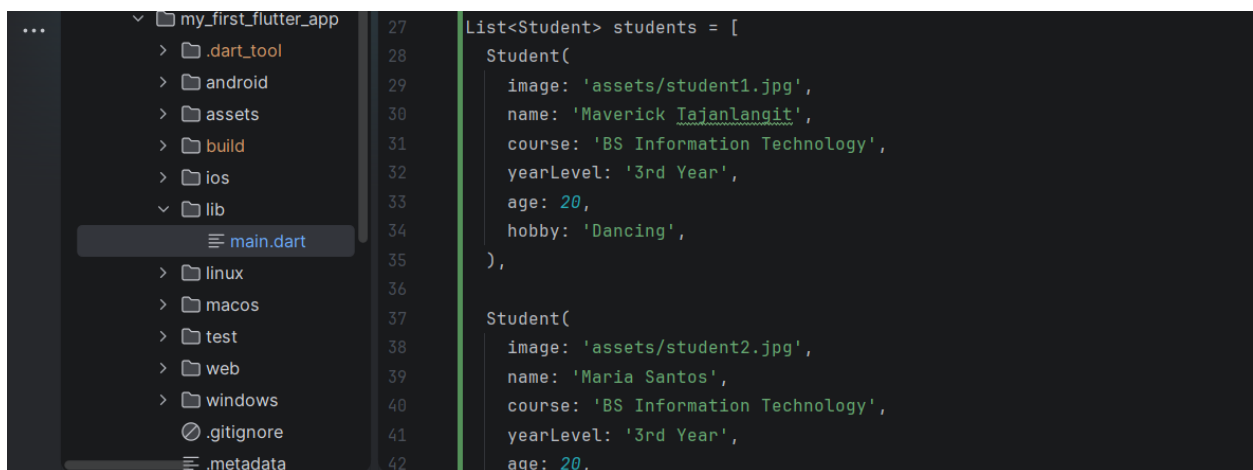
- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

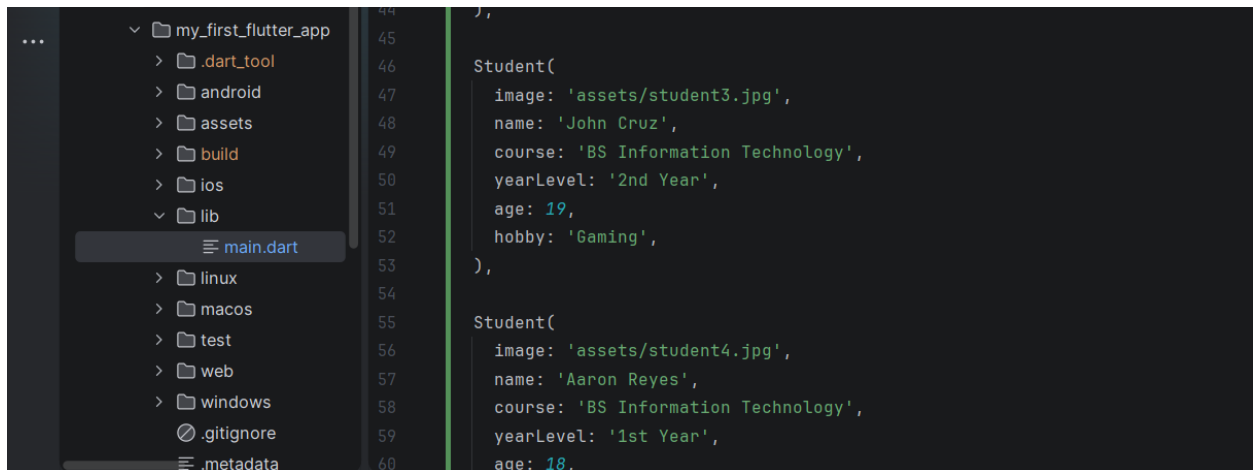
Screenshot:

Code:

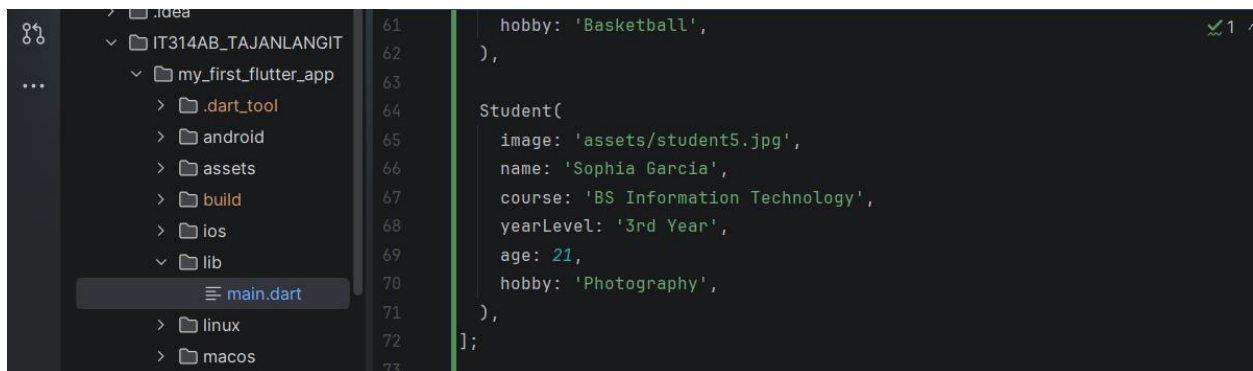
- Your student list
- At least five student entries



```
27 List<Student> students = [  
28   Student(  
29     image: 'assets/student1.jpg',  
30     name: 'Maverick Tajanlangit',  
31     course: 'BS Information Technology',  
32     yearLevel: '3rd Year',  
33     age: 20,  
34     hobby: 'Dancing',  
35   ),  
36  
37   Student(  
38     image: 'assets/student2.jpg',  
39     name: 'Maria Santos',  
40     course: 'BS Information Technology',  
41     yearLevel: '3rd Year',  
42     age: 20,
```



```
44 },
45
46 Student(
47   image: 'assets/student3.jpg',
48   name: 'John Cruz',
49   course: 'BS Information Technology',
50   yearLevel: '2nd Year',
51   age: 19,
52   hobby: 'Gaming',
53 ),
54
55 Student(
56   image: 'assets/student4.jpg',
57   name: 'Aaron Reyes',
58   course: 'BS Information Technology',
59   yearLevel: '1st Year',
60   age: 18,
```



```
61   hobby: 'Basketball',
62 ),
63
64 Student(
65   image: 'assets/student5.jpg',
66   name: 'Sophia Garcia',
67   course: 'BS Information Technology',
68   yearLevel: '3rd Year',
69   age: 21,
70   hobby: 'Photography',
71 ),
72 ];
73
```

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

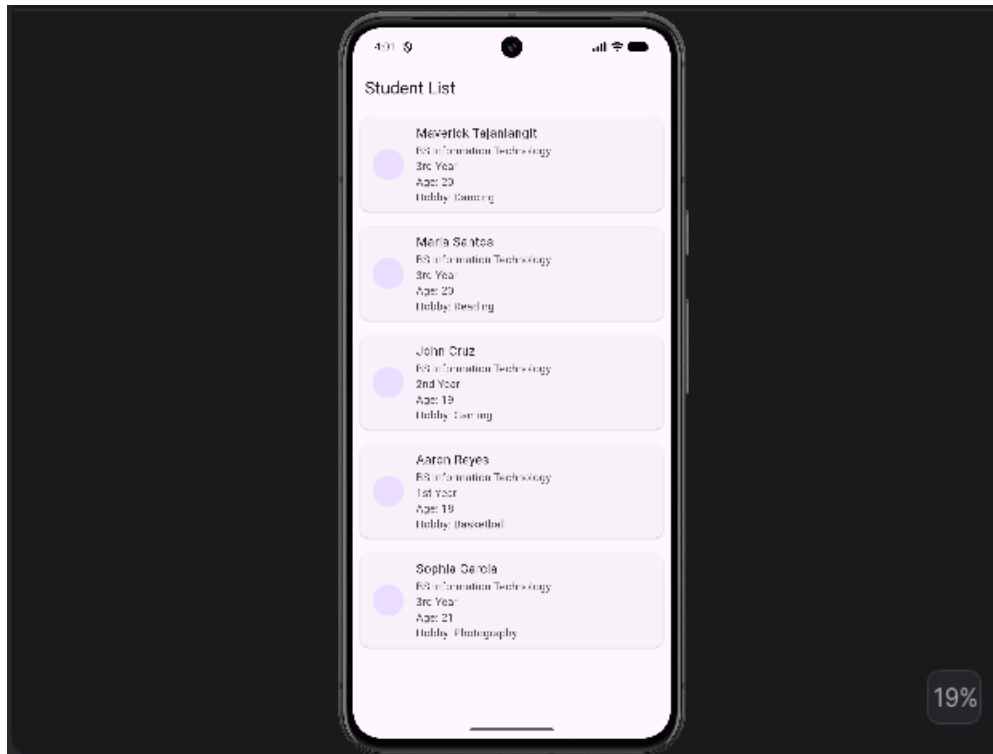
- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)



🚩 Flag 3 — Discover **ListView**

Objective: Make your student directory scrollable using **ListView**.

Research Flutter's **ListView**.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

Test

The user should be able to:

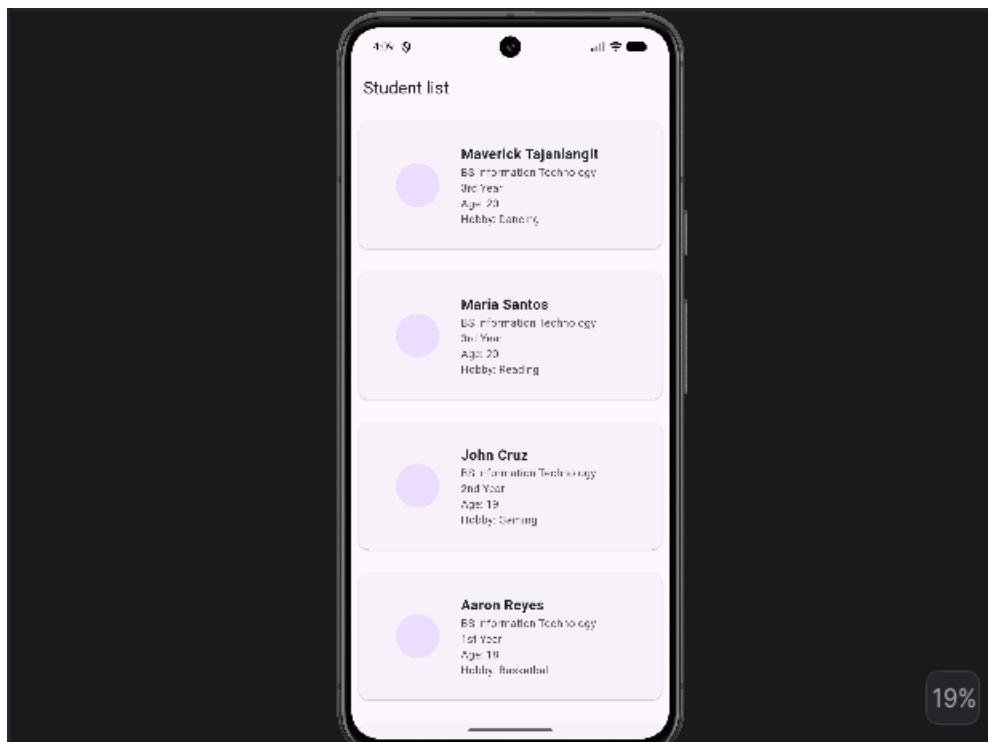
- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

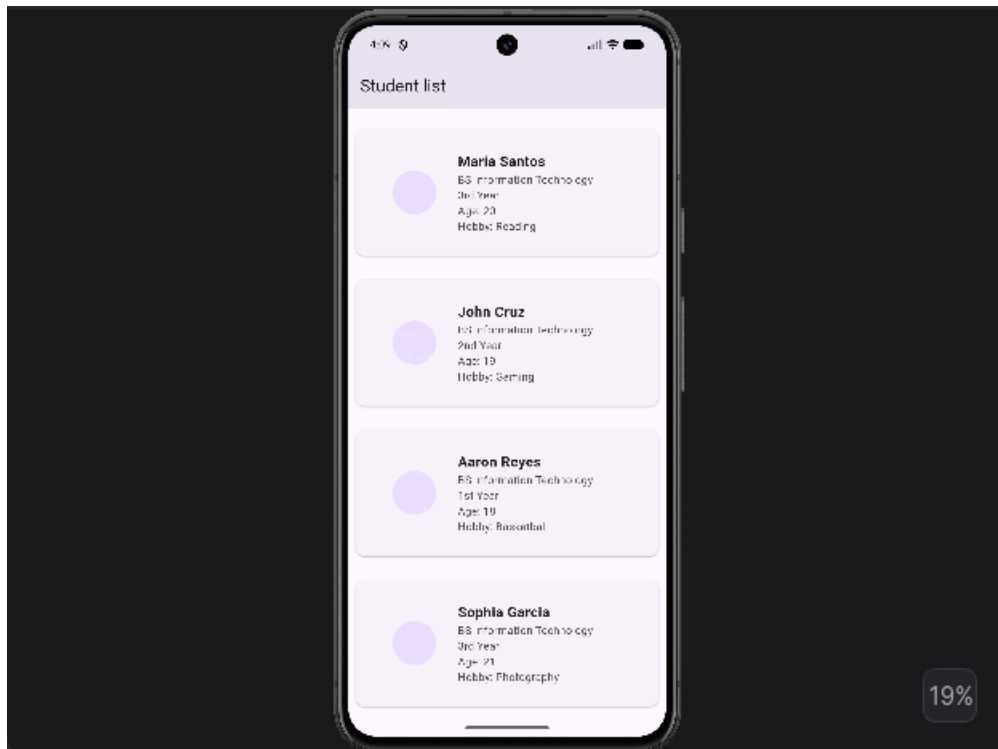
Screenshot:

- **ListView** in your code
- Top of the application
- Bottom portion showing later students



```
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
body: ListView(  
  children: [  
  
    Card(  
      margin: const EdgeInsets.symmetric(  
        horizontal: 10,  
        vertical: 15,  
      ),  
      child: Padding(  
        padding: const EdgeInsets.all(20),  
        child: ListTile(  
          title: Text(student.name),  
          subtitle: Text(student.major),  
          trailing: Text(student.year),  
          leading: CircleAvatar(  
            radius: 25,  
            backgroundImage: AssetImage('assets/profile.png'),  
          ),  
        ),  
      ),  
  ],  
)
```





🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

Test

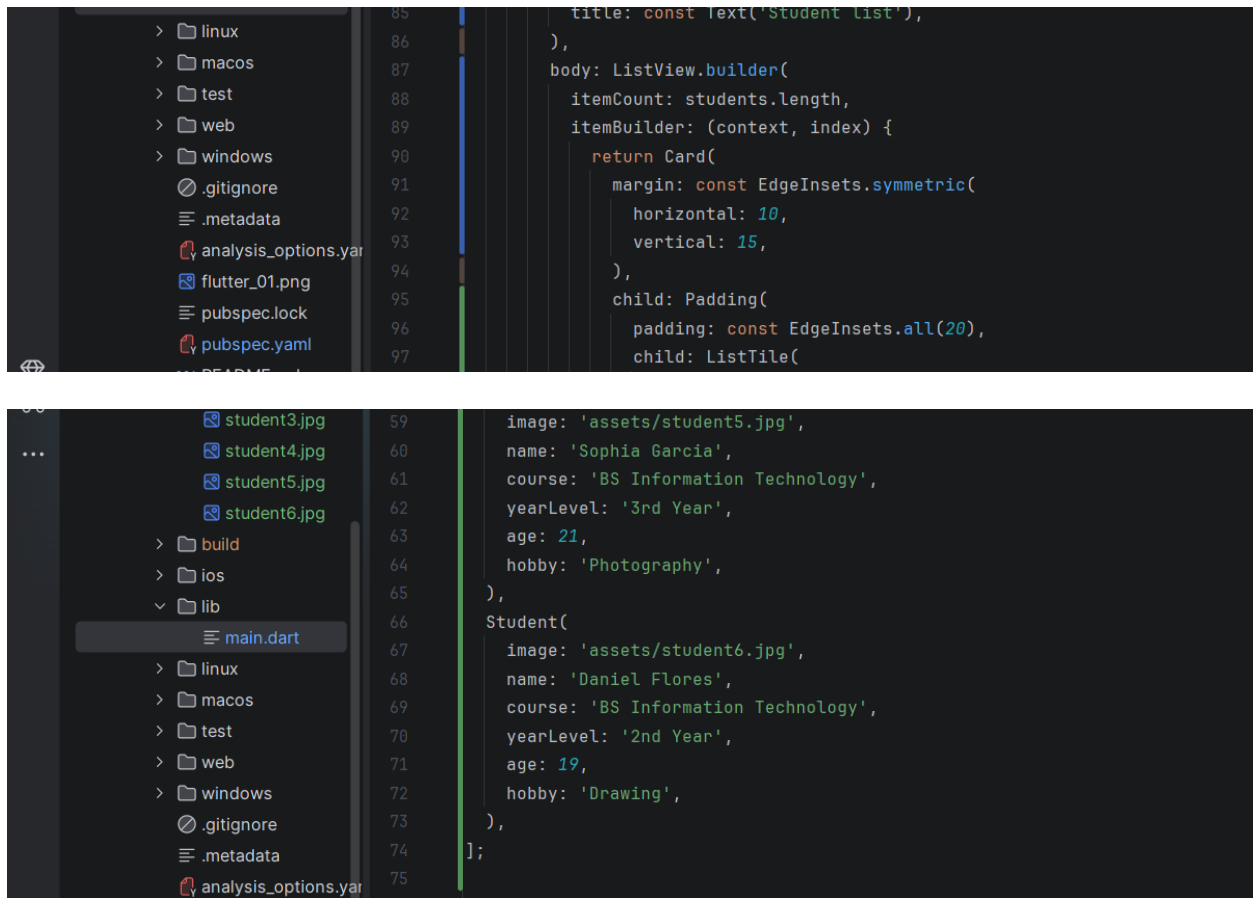
Add one new student to your list.

Without creating another card manually, your application should display the new student automatically.

Screenshot:

- `ListView.builder` in your code
- Student list

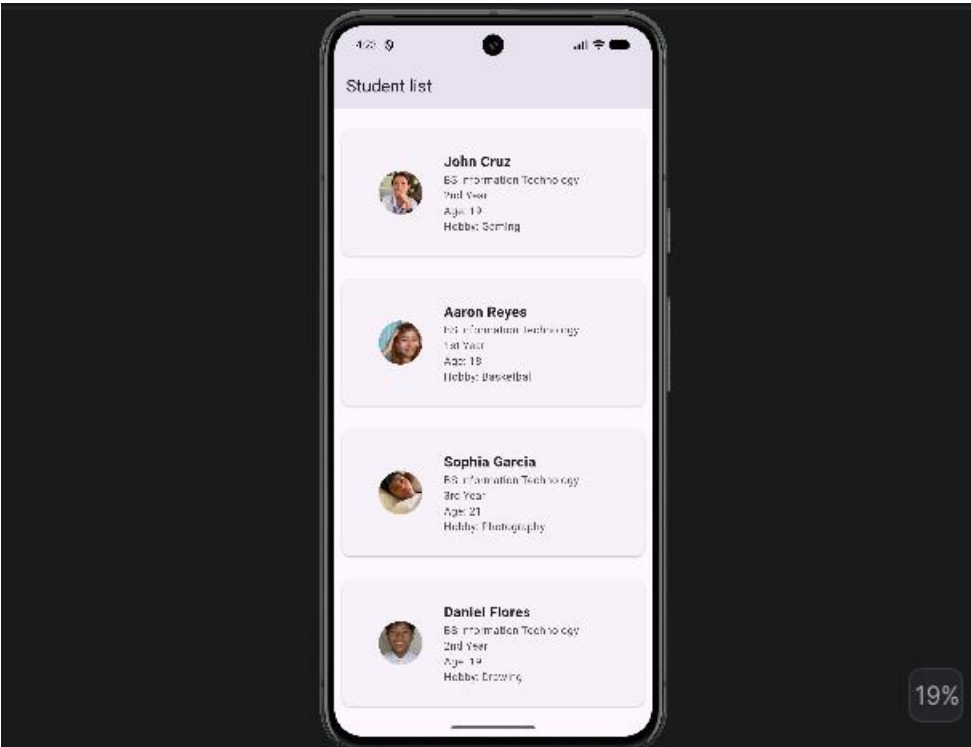
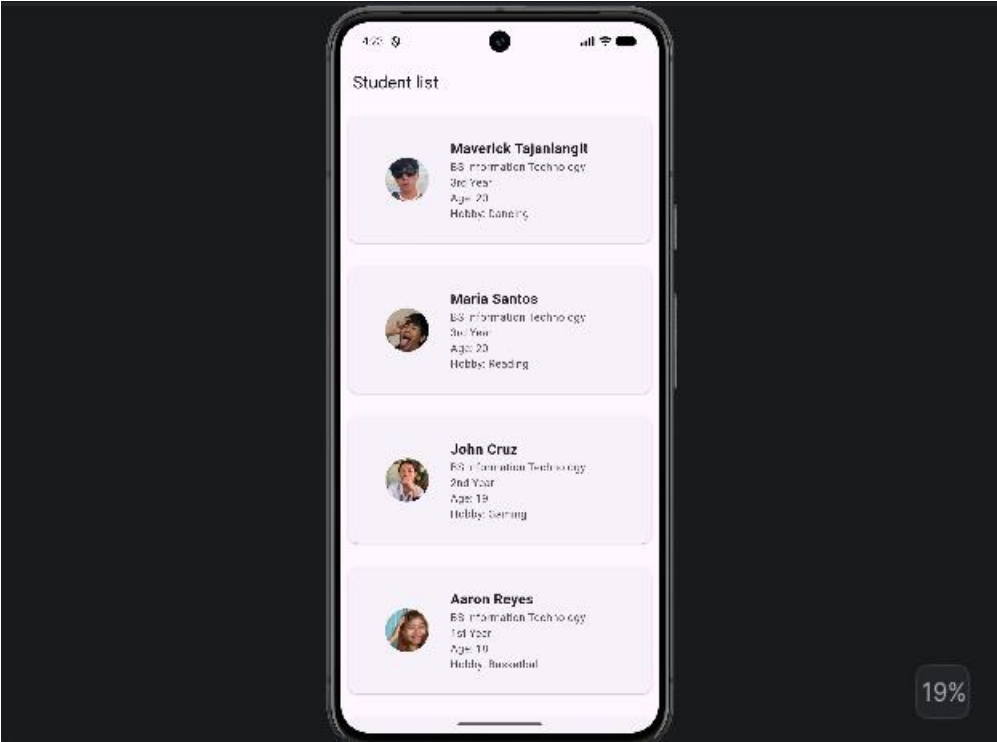
- Newly generated student appearing in the app



```
85 title: const Text('Student List'),
86 ),
87 body: ListView.builder(
88   itemCount: students.length,
89   itemBuilder: (context, index) {
90     return Card(
91       margin: const EdgeInsets.symmetric(
92         horizontal: 10,
93         vertical: 15,
94       ),
95       child: Padding(
96         padding: const EdgeInsets.all(20),
97         child: ListTile(
98           image: 'assets/student5.jpg',
99           name: 'Sophia Garcia',
100          course: 'BS Information Technology',
101          yearLevel: '3rd Year',
102          age: 21,
103          hobby: 'Photography',
104        ),
105        Student(
106          image: 'assets/student6.jpg',
107          name: 'Daniel Flores',
108          course: 'BS Information Technology',
109          yearLevel: '2nd Year',
110          age: 19,
111          hobby: 'Drawing',
112        ),
113      ],
114    );
```

The file explorer on the left shows the following structure:

- linux
- macos
- test
- web
- windows
- .gitignore
- .metadata
- analysis_options.yaml
- flutter_01.png
- pubspec.lock
- pubspec.yaml
- assets
 - student3.jpg
 - student4.jpg
 - student5.jpg
 - student6.jpg
- build
- ios
- lib
 - main.dart



🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

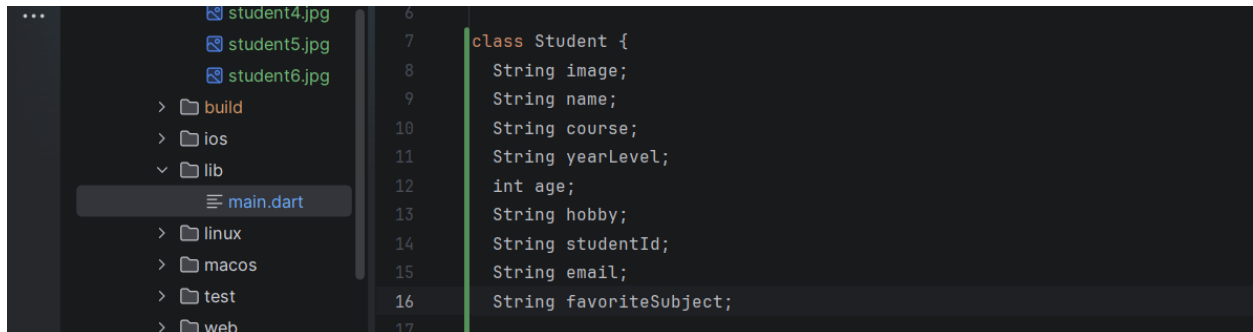
Examples:

- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

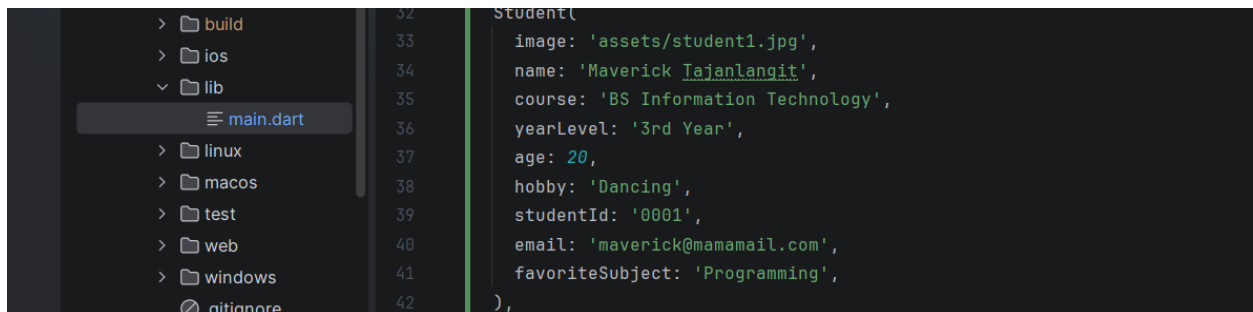
Screenshot:

- Updated student data
- Student Card displaying the additional fields



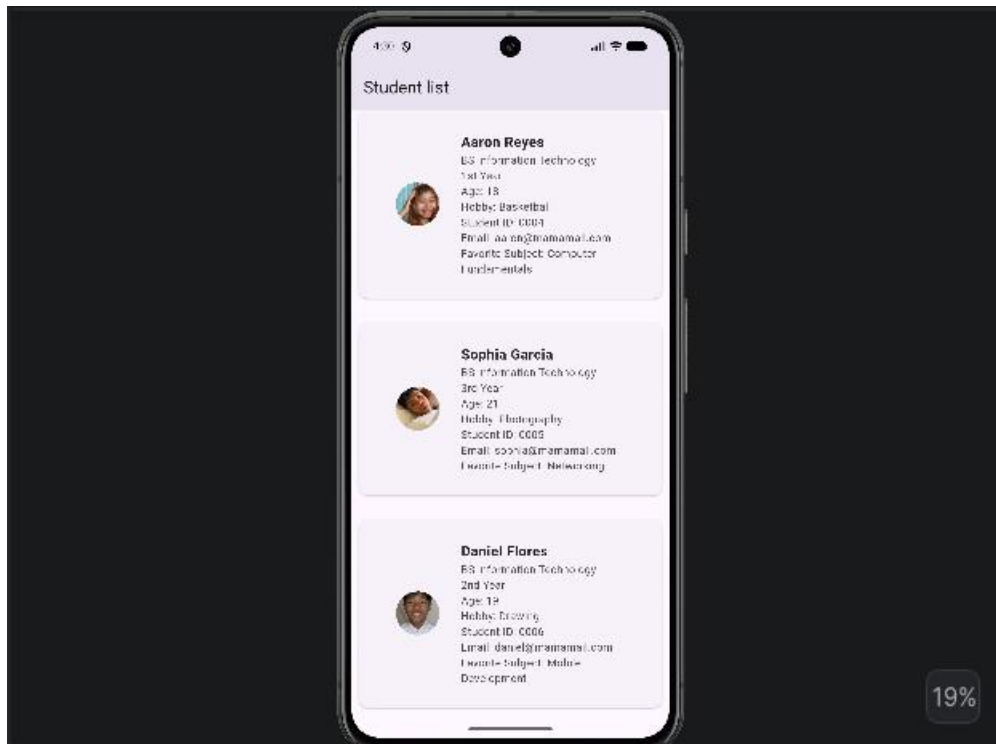
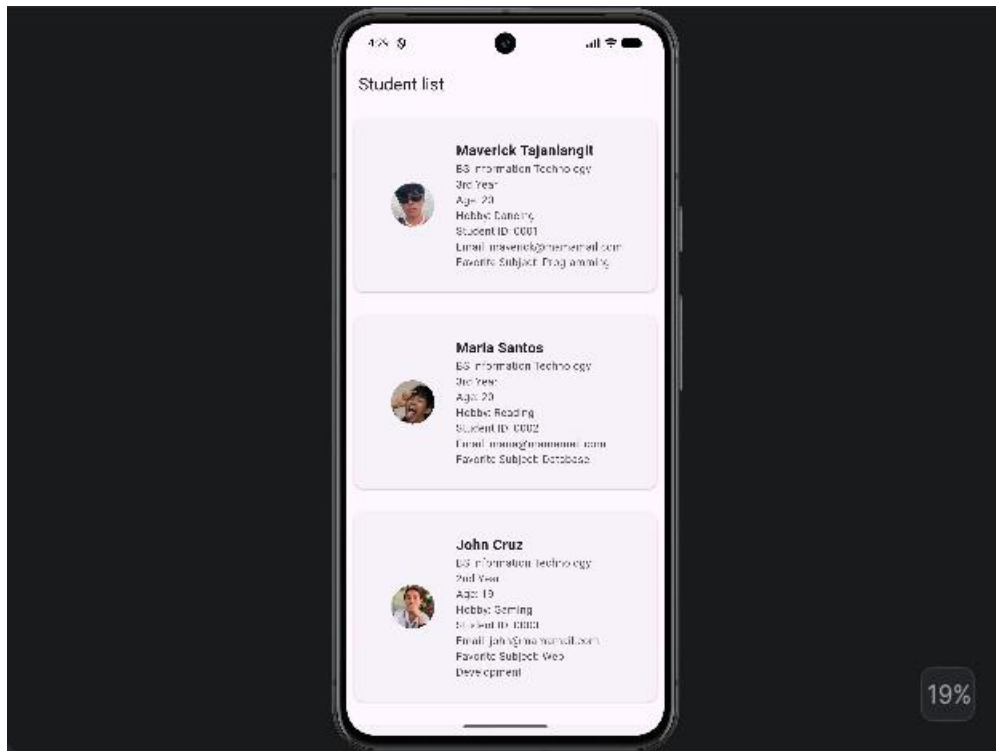
```
... student4.jpg 6
student5.jpg 7
student6.jpg 8
> build 9
> ios 10
< lib 11
  main.dart 12
  linux 13
  macos 14
  test 15
  web 16
17

class Student {
  String image;
  String name;
  String course;
  String yearLevel;
  int age;
  String hobby;
  String studentId;
  String email;
  String favoriteSubject;
}
```



```
> build 32
> ios 33
< lib 34
  main.dart 35
  linux 36
  macos 37
  test 38
  web 39
  windows 40
  .gitignore 41
42

Student(
  image: 'assets/student1.jpg',
  name: 'Maverick Tajanlangit',
  course: 'BS Information Technology',
  yearLevel: '3rd Year',
  age: 20,
  hobby: 'Dancing',
  studentId: '0001',
  email: 'maverick@mamamail.com',
  favoriteSubject: 'Programming',
),
```



🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Imagine your app just connected to a database.

The database returns:

```
[]
```

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

- "No students found."
- "Student list is empty."

Test

- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

Screenshot:

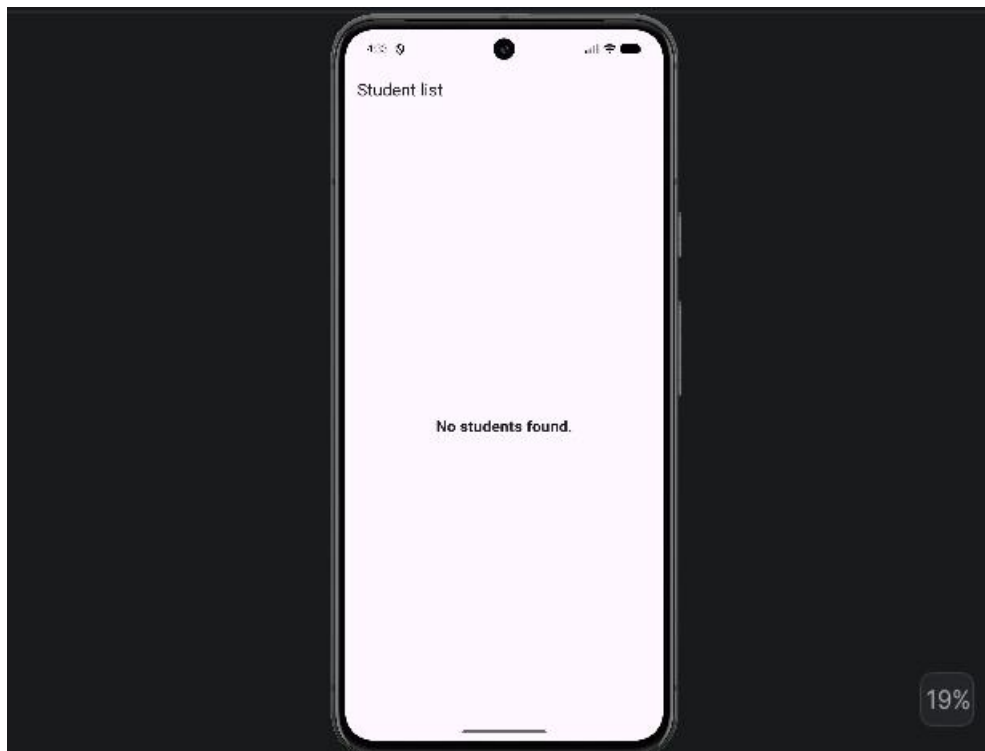
- Empty student list
- Empty-state UI
- Application still functioning correctly

```
... student4.jpg
student5.jpg
student6.jpg
> build
> ios
v lib
  main.dart
> linux
> macos
> test
> web
> windows

26 required this.email,
27 required this.favoriteSubject,
28 });
29 }
30
31 List<Student> students = [];
32
33 class MyApp extends StatelessWidget {
34   const MyApp({super.key});
35
36   @override
37   Widget build(BuildContext context) {
```

```
> build
> ios
v lib
  main.dart
> linux
> macos
> test
> web
> windows
.gitignore

41 appBar: AppBar(
42   title: const Text('Student list'),
43 ),
44 body: students.isEmpty
45   ? const Center(
46     child: Text(
47       'No students found.',
48       style: TextStyle(
49         fontSize: 20,
50         fontWeight: FontWeight.bold,
```



🚩 Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a `List`.

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

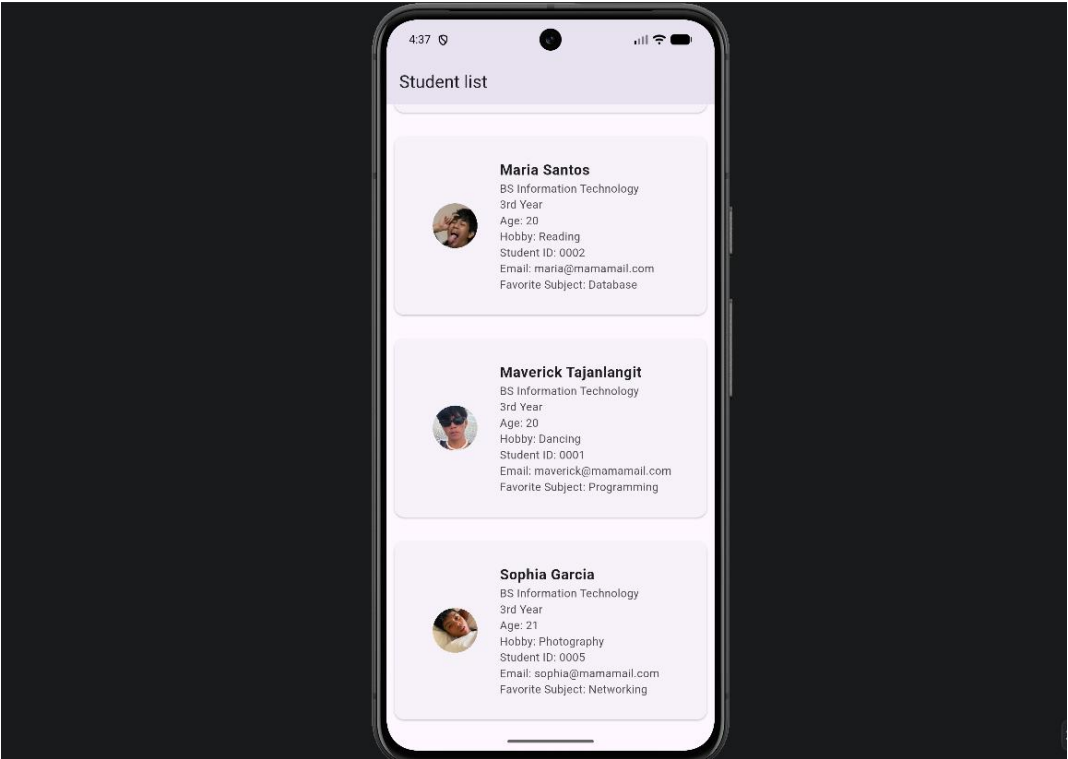
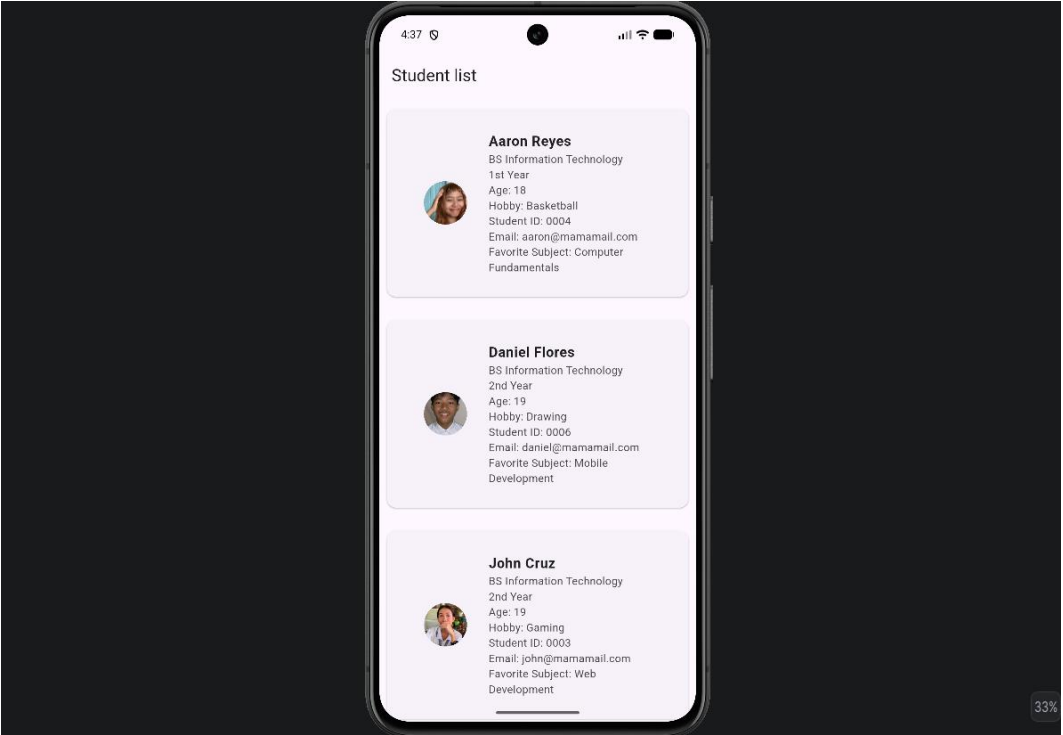
After sorting:

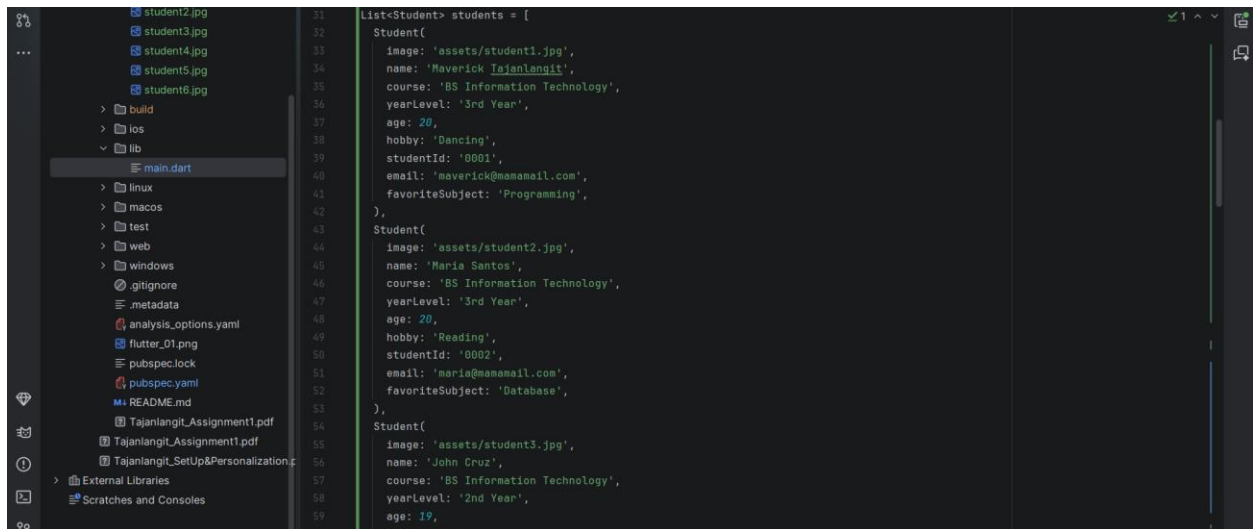
- Aaron
- John
- Maria
- Sophia

Your UI should immediately reflect the new order.

Screenshot:

- Student list before or after sorting
- Application displaying the sorted order





🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

feature/student-list

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
PS C:\Users\admin\Documents\FlutterProjects\Student-List> git commit -m "feat: CTF4 student list"
[feature/student-list (root-commit) b61718d] feat: CTF4 student list
8 files changed, 2396 insertions(+)
create mode 100644 .idea/.gitignore
create mode 100644 .idea/Student-List.iml
create mode 100644 .idea/caches/deviceStreaming.xml
create mode 100644 .idea/deviceManager.xml
create mode 100644 .idea/misc.xml
create mode 100644 .idea/modules.xml
create mode 100644 .idea/vcs.xml
create mode 160000 IT314AB_TAJANLANGIT
PS C:\Users\admin\Documents\FlutterProjects\Student-List> |
```